

DB接続ガイド — モックデータをMySQLに繋ぎ変える方法

このアプリは現在、**データベースに接続せず**、`dao` フォルダの中に書かれた「仮データ（モックデータ）」だけで動いています。このドキュメントでは「①どこにデータの取得処理が書いてあるか」「②それを実際のMySQL接続に書き換えるには何をすればよいか」を説明します。

1. 「DB接続部分」はどこにあるか

データを取得・登録・更新・削除する処理は、すべて `src/main/java/dao/` フォルダの中の5つのファイルに集約されています。

```
dao/
├─ AccountsDAO.java      ... アカウント（ログインユーザー）情報
├─ ReservationsDAO.java  ... 座席予約情報
├─ SeatsDAO.java         ... 座席マスタ情報
├─ LocationsDAO.java     ... 拠点（オフィス）マスタ情報
└─ MastersDAO.java       ... マスタ更新画面用（座席の追加・編集・削除）
```

****「DBに繋ぐ」 = 「この5つのファイルの中身を、SQLを実行するコードに書き換える」****ということになります。他のファイル（Logic・JSP・entity）は一切変更不要です。これがDAOで処理を分けている最大のメリットです。

目印は `// TODO: 後でMySQL接続に差し替え`

各DAOのメソッドの中に、次のようなコメントが必ず書いてあります。

```
// TODO: 後でMySQL接続に差し替え: SELECT * FROM accounts WHERE user_id = ?
public Account findById(String userId) {
    return ACCOUNTS.stream()
        .filter(a -> a.getUserId().equals(userId))
        .findFirst().orElse(null);
}
```

このコメントの**右側にSQL文のヒントも書いてあります**。「このメソッドは、こういうSQLを実行する想定で作った」という設計メモです。コメントが付いている行から始まるメソッドの中身を、丸ごとJDBCのコードに置き換えます。

2. 接続までの全体準備（最初の1回だけ）

① MySQLにテーブルを作る

`entity` クラスのフィールド（`AccountsDAO` などのモックデータ）を参考に、対応するテーブルを作成します。例えば `Account` なら：

```
CREATE TABLE accounts (  
    user_id    VARCHAR(20) PRIMARY KEY,  
    password   VARCHAR(50) NOT NULL,  
    name       VARCHAR(50) NOT NULL,  
    b_id       INT          NOT NULL,  
    admin      INT          NOT NULL DEFAULT 0  
);
```

同じ要領で `reservations` ・ `seats` ・ `locations` のテーブルも、各 `entity` クラスのフィールドを見ながら作成します。

② JDBCドライバ（MySQL Connector/J）を追加する

EclipseでMySQLに接続するには、「MySQL用の橋渡し役のライブラリ（JDBCドライバ）」が必要です。

1. [MySQL公式サイト](#) から `mysql-connector-j-x.x.x.jar` をダウンロード
2. `WebContent/WEB-INF/lib/` フォルダにコピー
3. Eclipseでプロジェクトを右クリック → **Properties** → **Java Build Path** → **Libraries** → **Add JARs...** で追加

③ 接続情報をまとめる共通クラスを作る（推奨）

毎回「接続文字列・ユーザー名・パスワード」を書くと修正が大変なので、共通の接続クラスを1つ作っておくと楽になります。

```
package dao;

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;

public class DBUtil {

    private static final String URL = "jdbc:mysql://localhost:3306/za_navi_db?useSSL=false&serverTimezone=Asia/Tokyo&characterEncoding=UTF-8";

    private static final String USER = "root";

    private static final String PASS = "your_password";

    static {
        try {
            // ドライバを明示的に読み込んでDriverManagerに登録する
            Class.forName("com.mysql.cj.jdbc.Driver");
        } catch (ClassNotFoundException e) {
            throw new RuntimeException("MySQL JDBCドライバが見つかりません。jarファイルを追加したか確認してください。", e);
        }
    }

    public static Connection getConnection() throws SQLException {
        // 登録済みのドライバの中から、URLの "jdbc:mysql://" に対応するものをDriverManagerが選んで接続する
        return DriverManager.getConnection(URL, USER, PASS);
    }
}
```

⚠ パスワードを直接書くのは学習用の簡易対応です。実務では設定ファイルや環境変数に分離します。

3. 実際の書き換え例（AccountsDAO#findById）

書き換え前（モック）

```
// TODO: 後でMySQL接続に差し替え: SELECT * FROM accounts WHERE user_id = ?

public Account findById(String userId) {
    return ACCOUNTS.stream()
        .filter(a -> a.getUserId().equals(userId))
        .findFirst().orElse(null);
}
```

書き換え後（MySQL接続）

```
public Account findById(String userId) {
    String sql = "SELECT * FROM accounts WHERE user_id = ?";
    try (Connection conn = DBUtil.getConnection();
        PreparedStatement ps = conn.prepareStatement(sql)) {

        ps.setString(1, userId);                // ㉑ ?の部分に値をセット
        ResultSet rs = ps.executeQuery();        // ㉒ SQLを実行して結果を取得

        if (rs.next()) {                        // ㉓ 結果が1件あれば
            return new Account(
                rs.getString("user_id"),
                rs.getString("password"),
                rs.getString("name"),
                rs.getInt("b_id"),
                rs.getInt("admin")
            );
        }
        return null;                            // ㉔ 該当なしならnull
    } catch (SQLException e) {
        e.printStackTrace();
        return null;
    }
}
```

書き換えの「型」はいつも同じ

どのDAOメソッドも、書き換えの基本パターンは共通です。

手順	やること	コード
①	SQL文を用意する	<code>String sql = "SELECT ... WHERE ... = ?";</code>
②	コネクションとステートメントを取得する	<code>try (Connection conn = DBUtil.getConnection(); PreparedStatement ps = conn.prepareStatement(sql)) {</code>
③	? に値をセットする	<code>ps.setString(1, userId);</code>
④	SQLを実行する	検索なら <code>executeQuery()</code> 、登録/更新/削除なら <code>executeUpdate()</code>
⑤	結果を取り出してentityに詰め替える（検索の場合）	<code>rs.getString("列名")</code> などでentityを組み立てる
⑥	例外処理をする	<code>catch (SQLException e) { ... }</code>

一覧を返すメソッド（例: `findAll()`）の場合は、`if (rs.next())` の代わりに `while (rs.next())` でループしながらリストに詰めていく形になります。

```
public List<Account> findAll() {
    List<Account> list = new ArrayList<>();
    String sql = "SELECT * FROM accounts";
    try (Connection conn = DBUtil.getConnection();
        PreparedStatement ps = conn.prepareStatement(sql);
        ResultSet rs = ps.executeQuery()) {

        while (rs.next()) {
            list.add(new Account(
                rs.getString("user_id"), rs.getString("password"),
                rs.getString("name"), rs.getInt("b_id"), rs.getInt("admin")
            ));
        }
    } catch (SQLException e) {
        e.printStackTrace();
    }
    return list;
}
```

登録・更新・削除（`INSERT` / `UPDATE` / `DELETE`）の場合は戻り値が `void` や `boolean` になり、`executeUpdate()` を使います。

```

public boolean insert(Account account) {
    String sql = "INSERT INTO accounts (user_id, password, name, b_id, admin) VALUES (?, ?, ?, ?, ?)";
    try (Connection conn = DBUtil.getConnection();
        PreparedStatement ps = conn.prepareStatement(sql)) {

        ps.setString(1, account.getUserId());
        ps.setString(2, account.getPassword());
        ps.setString(3, account.getName());
        ps.setInt(4, account.getBId());
        ps.setInt(5, account.getAdmin());

        return ps.executeUpdate() > 0;    // 何件更新されたかが返ってくる

    } catch (SQLException e) {
        e.printStackTrace();
        return false;
    }
}

```

4. 切り替え作業の進め方（おすすめの順番）

1. **まずは1つのDAO・1つのメソッドだけ書き換えてみる**（例: `AccountsDAO#findById`）
2. ログイン画面で動作確認 → 正しくMySQLのデータでログインできるか確認
3. 慣れてきたら他のメソッド・他のDAOへ広げていく
4. すべて書き換えたら、モックデータを保持していたリスト（`Arrays.asList(...)` で作られた `ACCOUNTS` や `SEATS` などの `static final List<...>` フィールド）は削除してOK ※ `OFFICE_IMAGES` のような `static { ... }` ブロックは「拠点名→画像ファイル名」の対応表であり、モックデータではないので **削除しないこと**（DB接続後も使い続ける）

一度に全部書き換えようとせず、1メソッドずつ「動く状態」を保ちながら進めるのがコツです。途中でエラーが出ても、どこを直したから動かなくなったのかが分かりやすくなります。

注意：1つの画面が複数のDAOメソッドを組み合わせて使っている場合がある

例えば氏名検索・部門検索や、ログイン後のメニュー画面（同じ部署の出社人数・2週間分の予約状況など）は、

AccountsDAO（ユーザー情報・部署メンバー）

→ ReservationsDAO（その人の本日・期間中の予約）

→ SeatsDAO（予約している座席の詳細）

のように、**複数のDAOのメソッドをリレー形式で呼び出して**1つの画面を作っています。このような画面では、AccountsDAO だけMySQL接続に切り替えて ReservationsDAO がまだモックのまま、という状態だと「実際にはいないはずのモックユーザーの予約が表示される」など、データの整合性が取れずに動作確認がしづらくなります。

1つの画面に関わるDAOは、できるだけまとめて切り替えるのがおすすめです。どのDAOがどこで使われているか分からなくなったら、Eclipseの **検索 → ファイル内検索（Ctrl+H）** で `new AccountsDAO()` のような文字列を検索すると、使用箇所が一覧で確認できます。

5. つまづきやすいポイント

症状	原因と対処
<code>ClassNotFoundException: com.mysql.cj.jdbc.Driver</code>	JDBCドライバ（jarファイル）が読み込まれていない。 <code>WEB-INF/lib</code> に入れたか、Build Pathに追加したか確認
<code>Communications link failure</code>	MySQLサーバーが起動していない、またはURLのポート番号・DB名が間違っている
<code>Access denied for user</code>	ユーザー名・パスワードが間違っている
日本語が文字化けする	接続URLに <code>characterEncoding=UTF-8</code> が付いているか確認、テーブルの文字コードが <code>utf8mb4</code> になっているか確認
<code>SQLException</code> が握りつぶされて原因が分からない	<code>catch</code> の中に <code>e.printStackTrace();</code> を入れて、Eclipseのコンソールに出力されるログを確認する

関連ドキュメント:

- [01 プログラム解説書.md](#) — アプリ全体の構造と各ファイルの役割
- [03 研修ハンズオン手順書.md](#) — 研修当日の手順まとめ